Chapitre 6 – Gestion d'état avancée

6.1 – Introduction

Le cœur de Jetpack Compose, c'est **l'état**.
Compose redessine automatiquement l'UI quand l'état change.
Mais dans une vraie application, il faut savoir :

- **où stocker l'état**,
- **comment le partager entre plusieurs Composables**,
- **comment le préserver (ex : rotation d'écran)**.

👉 Ce chapitre t'apprend à gérer l'état **de façon propre, performante et persistante**, avec :

- remember et mutableStateOf,
- rememberSaveable,
- derivedStateOf,
- LaunchedEffect et SideEffect,
- ViewModel,
- LiveData et StateFlow.

6.2 – Rappel : UI = fonction de l'état

En Compose, une UI dépend uniquement de l'état courant :

```
@Composable
fun CounterApp() {
    var count by remember { mutableStateOf(0) }

    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        Text("Compteur : $count")
        Button(onClick = { count++ }) { Text("Incrémenter") }
    }
}
```

Quand count change → Compose **recompose** la fonction → la nouvelle valeur est affichée.

6.3 – remember vs rememberSaveable

- remember : garde l'état tant que le Composable est en mémoire.
- rememberSaveable : garde l'état **même après rotation d'écran ou recréation de l'activité**.

Exemple :

```
@Composable
fun RememberExample() {
    var text by rememberSaveable { mutableStateOf("") }

    TextField(
        value = text,
        onValueChange = { text = it },
        label = { Text("Entrez du texte") }
    )
}
```

👉 Utilise rememberSaveable pour tout état que tu veux **préserver dans le temps**.

6.4 – derivedStateOf

Permet de **dériver un nouvel état** à partir d'un autre sans recalcul inutile.

Exemple :

```
@Composable
fun DerivedExample() {
    var text by remember { mutableStateOf("") }
    val isLong = remember { derivedStateOf { text.length > 10 } }

    Column {
        TextField(value = text, onValueChange = { text = it })
        Text(if (isLong.value) "Texte long" else "Texte court")
    }
}
```

derivedStateOf évite que Compose redessine inutilement quand la valeur dérivée ne change pas.

6.5 – Side effects : LaunchedEffect, SideEffect, DisposableEffect

🔹 **LaunchedEffect**

Permet d'exécuter du code **une seule fois** ou **quand une clé change**.

```
@Composable
fun TimerExample() {
    var seconds by remember { mutableStateOf(0) }
```

```
    LaunchedEffect(Unit) {
        while (true) {
            delay(1000)
            seconds++
        }
    }

    Text("Temps écoulé : $seconds s")
}
```

👉 Idéal pour lancer une coroutine liée au cycle de vie du Composable.

SideEffect

Exécute du code après chaque recomposition (utile pour logs ou analytics).

```
SideEffect {
    Log.d("Compose", "UI recomposée")
}
```

DisposableEffect

Permet de nettoyer une ressource quand le Composable est retiré de la composition.

```
DisposableEffect(Unit) {
    Log.d("Compose", "Initialisation")
    onDispose {
        Log.d("Compose", "Nettoyage")
    }
}
```

6.6 – Gestion d'état avec ViewModel

Compose est parfait pour les états **éphémères**, mais pas pour les états **durables** (ex : données utilisateur, listes, API).
C'est là que le **ViewModel** intervient.

Exemple simple

```
class CounterViewModel : ViewModel() {
    private val _count = mutableStateOf(0)
    val count: State<Int> = _count

    fun increment() {
        _count.value++
    }
}
```

Dans Compose :

```
@Composable
fun CounterScreen(viewModel: CounterViewModel = viewModel()) {
```

```
    val count by viewModel.count
    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        Text("Compteur : $count")
        Button(onClick = { viewModel.increment() }) { Text("Incrémenter") }
    }
}
```

✅ Ici, l'état est stocké **dans le ViewModel** : il survit à la rotation et reste séparé de l'UI.

6.7 – ViewModel + Flow / LiveData

Le ViewModel peut exposer un **Flux de données réactif** (Flow ou LiveData).

Exemple avec Flow

```
class UserViewModel : ViewModel() {
    private val _username = MutableStateFlow("Inconnu")
    val username = _username.asStateFlow()

    fun updateName(newName: String) {
        _username.value = newName
    }
}
```

Côté Compose :

```
@Composable
fun UserScreen(viewModel: UserViewModel = viewModel()) {
    val name by viewModel.username.collectAsState()
    Column {
        Text("Nom : $name")
        Button(onClick = { viewModel.updateName("Alice") }) {
            Text("Changer le nom")
        }
    }
}
```

👉 collectAsState() convertit un Flow en un State observable par Compose.

6.8 – Exercices pratiques

📝 Exercice 1 : rememberSaveable

Crée un champ texte dont la valeur reste même après rotation d'écran.

📝 Exercice 2 : derivedStateOf

Crée un champ texte et affiche "Texte long" si plus de 15 caractères, sinon "Texte court".

📝 **Exercice 3 : LaunchedEffect**

Affiche un compteur qui augmente automatiquement toutes les secondes.

📝 **Exercice 4 : ViewModel**

Crée un CounterViewModel avec increment() et reset(), et connecte-le à un Composable CounterUI.

📝 **Exercice 5 : Flow**

Crée un ViewModel qui expose un StateFlow<String> et un Composable qui affiche la valeur.

6.9 – Défi pratique

Crée une **application Compose + ViewModel** de "Notes rapides" :

- Un champ TextField pour écrire une note.
- Un bouton "Ajouter".
- Une LazyColumn pour afficher la liste des notes.
- Les notes sont conservées dans un ViewModel.
- Quand tu quittes et reviens, elles doivent rester (simulation avec rememberSaveable).

👉 Objectif : combiner ViewModel, StateFlow, et rememberSaveable pour une gestion d'état robuste.

6.10 – Quiz de fin de chapitre

1. Quelle est la différence entre remember et rememberSaveable ?
    - a) rememberSaveable garde l'état après rotation.
    - b) remember sauvegarde automatiquement dans le ViewModel.
    - c) Aucune différence.
2. Que fait derivedStateOf ?

    - a) Calcule un nouvel état à partir d'un autre sans recomposition inutile.
    - b) Remplace rememberSaveable.
    - c) Supprime un état.
3. Vrai ou Faux : LaunchedEffect permet d'exécuter du code suspendu dans un Composable.
4. Quelle fonction permet de transformer un Flow en State dans Compose ?
    - a) observeAsState()
    - b) collectAsState()
    - c) flowToState()
5. Pourquoi utiliser un ViewModel ?
    - a) Pour stocker les états durables indépendamment du cycle de vie.
    - b) Pour afficher du texte.
    - c) Pour gérer les couleurs.

6.11 – Correction du quiz

1. **a)** → rememberSaveable garde l'état après rotation.
2. **a)** → derivedStateOf dérive un état sans recomposer inutilement.
3. **Vrai** → LaunchedEffect lance une coroutine dans Compose.
4. **b)** → collectAsState() transforme un Flow en State.
5. **a)** → Le ViewModel garde l'état durablement.

✅ À la fin de ce chapitre, tu maîtrises la **gestion d'état réactive et avancée** dans Compose :

- remember, rememberSaveable
- derivedStateOf
- Effets secondaires (LaunchedEffect, SideEffect, DisposableEffect)
- Intégration complète du ViewModel et des Flow

👉 Le prochain chapitre (**Chapitre 7 – Navigation et architecture**) te montrera comment **organiser une application multi-écrans** en utilisant **Navigation Compose** et une **architecture MVVM complète** (ViewModel, repository, navigation).